

Module.symvers

Владислав Богданов

2026-06-24

Оглавление

1. Введение	1
2. Что внутри файла?	1
3. Зачем он нужен на практике?	1
4. Где его найти?	2
5. Как это выглядит в Makefile?	2
6. Частые проблемы	2

1. Введение

Module.symvers (Symbol Versions) — это глобальный реестр (словарь) всех функций и переменных ядра, которые разрешено использовать внешним модулям.

2. Что внутри файла?

Если открыть этот файл (он обычный текстовый), то увидим строки, разделенные табуляцией. Вот его типичная структура:

```
0x12345678 printk vmlinux EXPORT_SYMBOL
0x87654321 kmalloc vmlinux EXPORT_SYMBOL_GPL
0xabcdef99 my_custom_func my_driver_module EXPORT_SYMBOL DEFAULT
```

Каждая строка содержит 5 колонок:

- **CRC (Контрольная сумма):** 32-битное число (в hex). Это самое важное! Это хэш от сигнатуры функции (ее имени, типов аргументов и возвращаемого значения).
- **Имя символа:** Название функции или глобальной переменной.
- **Модуль:** Где определен символ. Если там *vmlinux*, значит, функция вшита в само ядро. Если имя другого модуля — значит, функция экспортируется из него.
- **Тип экспорта (License):** Например, *EXPORT_SYMBOL* (можно использовать всем) или *EXPORT_SYMBOL_GPL* (только для модулей с GPL-лицензией).
- **Пространство имен (Namespace):** (Появилось в относительно новых ядрах) Используется для группировки символов, чтобы избежать конфликтов имен.

3. Зачем он нужен на практике?

У файла две критически важные задачи:

1. Сборка внешних (out-of-tree) модулей

Когда пишем свой драйвер, не компилируем его вместе с ядром. Используем Makefile, который обращается к уже собранному ядру. Система сборки ядра (kbuild) читает Module.symvers, чтобы понять: "А какие функции вообще доступны в этом конкретном ядре?". Если вызвать в своем коде функцию, которой нет в Module.symvers, сборка выдаст ошибку: WARNING: "my_func" undefined!.

2. Защита от Kernel Panic (Проверка версий)

Это магия CRC. Допустим, в ядре 5.15 функция выглядела так: void my_func(int a). А в ядре 6.1 разработчики добавили аргумент: void my_func(int a, int b). Если собрать свой модуль под ядро 5.15, а потом попытаться загрузить его в ядро 6.1, произойдет катастрофа (Kernel Panic), потому что модуль положит в стек один аргумент, а ядро будет ждать два. Чтобы этого избежать, при сборке модуля kbuild смотрит в Module.symvers, вычисляет

CRC вызываемых функций и "вшивает" эти CRC в наш .ko файл. Когда мы загружаем модуль (insmod), ядро сверяет CRC из модуля со своим текущим Module.symvers. Если они не совпадают — ядро вежливо откажется грузить модуль с *ошибкой disagrees about version of symbol*. Система спасена!



Эта фича работает, если ядро собрано с включенной опцией CONFIG_MODVERSIONS=y, что в 99% дистрибутивов так и есть.

4. Где его найти?

Обычно он лежит в директории с заголовками и исходниками твоего текущего ядра:

```
/lib/modules/$(uname -r)/build/Module.symvers
```

Если сам компилировал ядро из исходников, он будет в корне дерева исходников ядра.

5. Как это выглядит в Makefile?

Когда пишешь Makefile для внешнего модуля, указываем путь к ядру (обычно через переменную KDIR или KERNELDIR). Система сборки kbuild автоматически подхватывает Module.symvers из этой директории. Не нужно прописывать его вручную, но нужно понимать, что он там используется.

```
# Простейший Makefile для внешнего модуля
obj-m += my_driver.o

# Указываем путь к исходникам ядра
KDIR := /lib/modules/$(shell uname -r)/build

all:
    make -C $(KDIR) M=$(PWD) modules

clean:
    make -C $(KDIR) M=$(PWD) clean
```

6. Частые проблемы

1. **Обновилось ядро, а заголовки нет.** Если в твоём Linux обновилось ядро, но ты не установил пакет linux-headers для новой версии, Module.symvers будет отсутствовать или будет от старой версии. Твой модуль либо не соберётся, либо упадет при загрузке. Всегда держи заголовки ядра синхронизированными с running kernel!
2. **Сторонние драйверы (Nvidia, VirtualBox).** Когда выходит новое ядро, драйверы от Nvidia или VirtualBox часто ломаются. Это происходит именно потому, что изменились CRC в Module.symvers, и старые бинарники модулей не проходят проверку версий.

Разработчикам приходится пересобирать их под новый Module.symvers.